Enunciado: Game Jam Java

Oct 30, 2026 · @Juan Manuel

El desafío

Tienen **4 horas** para construir, en equipo, un videojuego 2D en Java que funcione de punta a punta: menú, partida y pantalla de fin con el puntaje. Al terminar, cada equipo lo presenta y compite contra los demás.

No van a programar la ventana ni el bucle de juego: el docente les entrega un kit con eso, el teclado, el mouse y el reloj. **Todo lo demás lo programan ustedes:** las entidades, el mundo, las colisiones, la física, las reglas, las pantallas y las tres estructuras de datos. Ahí tienen que aplicar todo lo que vimos:

- **POO:** clases, objetos, encapsulamiento con validación.
- **Herencia y polimorfismo:** clases abstractas, subclases que se comportan distinto, sobreescritura y sobrecarga.
- **Interfaces:** contratos propios implementados por varias clases.
- **Paquetes y relaciones entre objetos:** un paquete por responsabilidad.
- **TADs:** su propia Lista, su propia Pila y su propia Cola, cada una con una función real dentro del juego.

Un juego simple y bien diseñado vale más que uno vistoso con el código desordenado. Gana el equipo que sume más puntos en la rúbrica de 100 puntos (más abajo). El voto del público suma 5; el diseño y las estructuras pesan mucho más.

Qué tiene que cumplir su juego

Para aprobar, el juego entregado tiene que cumplir **todo** esto:

- [ ] Compila de cero y se puede jugar: **menú, partida y pantalla de fin** que muestra el puntaje. No se guarda en ningún archivo; solo se ve en pantalla.
- [ ] Al menos **3 escenas** conectadas entre sí.
- [ ] Una **clase abstracta propia con 3 subclases concretas** que se comportan distinto (sobreescritura, polimorfismo real).
- [ ] Una **interfaz propia** implementada por 2 o más clases.
- [ ] **Encapsulamiento:** atributos `private`, constructores y setters que validan.

- [ ] **Una Lista, una Pila y una Cola propias**, que pasan el verificador del kit, y cada una **cumple una función real en su juego**. Una estructura que está pero no hace nada no cuenta.
- [ ] **Paquetes por responsabilidad**, sin dependencias circulares.
- [ ] Nada de colecciones de `java.util` (`ArrayList`, `LinkedList`, `Stack`, `Queue`, `Deque`, `Vector`, `Map`, `Set` y similares) ni de arreglos `[]` fuera de sus TADs.

Con eso aprueban. Para subir la nota: una segunda implementación de Lista con su justificación, sobrecarga de métodos con sentido, mecánicas propias (niveles, power-ups, dificultad creciente), pruebas propias y menos `instanceof` a favor de polimorfismo.

Elijan su juego

Cada equipo elige **uno** de los seis y se lo avisa al docente en los primeros 20 minutos. Los seis exigen las mismas tres estructuras; cambia el papel que cumple cada una.

#	Juego	Premisa	Lista	Cola	Pila	Dificultad
1	**Laboratorio de rebote**	Lanzan pelotas de distintos materiales (densidad y elasticidad) para embocarlas en un aro que se mueve	Pelotas y aro en el mundo	Cargador de pelotas por lanzar	Historial de aciertos	Media
2	**Snake con deshacer**	Una serpiente que crece al comer; con Z se deshace el último movimiento a cambio de puntos	Cuerpo de la serpiente (doblemente enlazada)	Giros pendientes	Movimientos para deshacer	Baja
3	**Breakout**	Pala, pelota y ladrillos de tres tipos; los power-ups se guardan y se usan en orden inverso	Ladrillos	Niveles en orden	Power-ups recogidos	Media
4	**Sokoban con deshacer**	Empujan cajas hasta las metas; Z deshace sin límite	Casillas y cajas	Niveles en orden	Movimientos para deshacer	Media

#	Juego	Premisa	Lista	Cola	Pila	Dificultad
5	**Restaurante frenético**	Los clientes llegan y hay que atenderlos antes de que se les acabe la paciencia	Mesas (circular)	Clientes en espera	Platos sucios	Media
6	**Defensa en el circuito**	Enemigos recorren un camino cerrado y las torres les disparan	Camino (circular)	Oleadas pendientes	Torres construidas	Alta

Cómo elegir

- **Si es su primera vez con un proyecto grande:** Snake o Laboratorio de rebote. Son los que antes se ponen a andar.
- **Si quieren física y choques:** Laboratorio de rebote y Breakout. En ambos la física y las colisiones las resuelven ustedes.
- **Si les gusta pensar el tablero:** Sokoban, donde la Pila de movimientos es el corazón del juego.
- **Si buscan un desafío:** Defensa en el circuito. Es el más ambicioso y exige tener bien organizado el equipo.

El detalle de cada juego (jerarquía de clases sugerida, mínimo y extras) está en la guía completa, sección **Los seis juegos**.

Condiciones

Equipos de 4 personas (3 si la cantidad no cierra). Cada integrante es dueño de un módulo y lo defiende solo en la exposición:

Rol	Paquete	Qué programa
A. Entidades	`juego.entidades`	La clase abstracta base de entidades y sus subclases: jugador, enemigos, ítems, obstáculos
B. Estructuras	`estructuras`	Su Lista, su Pila y su Cola, hasta que pasan el verificador
C. Reglas	`juego.reglas`	El mundo que recorre las entidades y la escena de la partida: colisiones, puntaje, vidas, fin del juego
D. Pantalla	`juego.pantallas`	Menú, pantalla de fin, HUD, textos y efectos

Restricciones

- Java 11 o superior y solo la librería estándar. `java.awt`, `javax.swing`, `java.util.Random` y `java.util.Iterator` están permitidos.
- No se modifican los paquetes del kit (`motor`, `contratos`, `ui`, `verificador` y las interfaces de `estructuras`). Si encuentran un error, avisen al docente.
- Las estructuras de datos del juego son las suyas. Los arreglos `[]` van solo dentro de sus implementaciones.
- Cualquier integrante tiene que poder explicar cualquier clase.

Qué reciben: la carpeta `kit-alumno` con la infraestructura (ventana, escenas, teclado y mouse, temporizador, dibujo de texto), las **interfaces** `Lista<T>`, `Pila<T>` y `Cola<T>` sin implementación, un verificador automático de sus TADs y una demo mínima. No reciben entidades, física, colisiones, HUD ni estructuras ya hechas.

Qué entregan a las 4 horas:

- La carpeta `src` comprimida, con el nombre del equipo, que compila de cero con `compilar.sh` o `compilar.bat`.
- Un diagrama de clases (a mano o en foto) con herencia, interfaces y relaciones.
- Una línea por integrante: nombre y paquete que programó.

Hitos: juego elegido y acta de contrato a los 0:20, esqueleto navegable a las 1:15, mínimo completo a las 2:45, congelamiento a las 3:30 y entrega a las 4:00. La demo (5 minutos) y la defensa (5 minutos) son después.

Cómo se evalúa

El proyecto vale **100 puntos**. El diseño y las estructuras pesan más del doble que la jugabilidad.

Criterio	Puntos
Diseño POO: herencia, abstractas, interfaces, encapsulamiento, polimorfismo real	30
TADs propios: pasan el verificador (10) y cumplen una función justificada en el juego (15)	25
Funcionalidad y jugabilidad	15
Defensa oral individual	15
Paquetes y relaciones entre objetos	10
Voto del público y creatividad	5

La defensa es individual: cada equipo hace 5 minutos de demo y 5 de preguntas, y las preguntas se reparten entre los integrantes sin importar quién escribió esa parte. Algunas que van a escuchar: ¿por qué usaron una Cola acá y no una Lista?, ¿qué clases hay que tocar para agregar un tipo nuevo de enemigo, pelota o ladrillo?, ¿por qué este atributo es `private` y qué estado imposible evita el setter?

Descuentos: si no compila de cero, 10 puntos menos. Por cada colección prohibida de `java.util` en la lógica del juego, 10 puntos menos; un arreglo `[]` fuera de un TAD para guardar elementos del juego cuenta como una.

Primeros 30 minutos

1. **Compilen y ejecuten la demo** (`demo.Main`) con `compilar.sh` o `compilar.bat`. El cuadrado se mueve con las flechas, el click cambia el color y ENTER cambia de escena. Si anda, el entorno está bien.
2. **Abran `verificador.PruebaMisEstructuras`.** Tiene las líneas comentadas: las activan cuando el equipo de Estructuras tiene su primera implementación. Hasta entonces no hay ninguna lista, pila ni cola disponible.
3. **Lean `demo/EscenaDemo.java` y `demo/EscenaDemoDos.java`.** Muestran cómo se escribe una escena, cómo se lee la entrada y cómo se pasa de una escena a otra.
4. **Elijan juego y repartan los módulos A, B, C y D.** Avisen al docente antes de los 20 minutos.
5. **Completen el acta de contrato:** una fila por cada clase que un módulo le pide a otro, con su responsable, su constructor y los métodos que usan los demás. Es lo que permite programar en paralelo sin romperse.

6. **Arranquen cada uno por su módulo,** programando contra las interfaces del acta. Estructuras entrega primero la `Lista`, porque el mundo depende de ella. Integren cada 45 a 60 minutos y compilen siempre de cero antes de juntar.

Todo el detalle (conceptos explicados con ejemplos, referencia de la API del kit, los seis juegos con jerarquía sugerida y errores comunes) está en la guía completa.